

```
Server/server_data_supabase.py
```

```
# server_data_supabase.py
```

```
# i o t o o i . i o E. o f < o " o (Supabase i o o - ERD E u < i i o )
```

```
import os
```

```
import sys
```

```
import json
```

```
from datetime import datetime
```

```
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
```

```
from supabase_client import supabase, SUPABASE_AVAILABLE
```

```
class ServerDataManager:
```

```
    """i o t o o i . i o E. o f < i . o i / (Supabase i < i ' - ERD E u < i i o )"""
```

```
    def __init__(self):
```

```
        self.supabase = supabase
```

```
        self.use_supabase = SUPABASE_AVAILABLE
```

```
    if not self.use_supabase:
```

```
        print("E t E : Supabase o %... i < i ' i i i i u o o /. JSON i i ... o " o o i i . o o - i ' o
```

```
        self._init_json_fallback()
```

```
    else:
```

```
        print("Supabase o o i . i o o t i . i / o " o o i i . E, o i o i i i u o o /. (ERD E u < i i o )")
```

```
    def _init_json_fallback(self):
```

```
        """JSON i i ... i . o o - o " o o i . E, o i """
```

```
        from env_utils import DATA_DIR
```

```
        self.folders_file = os.path.join(DATA_DIR, "folders.json")
```

```
        self.files_file = os.path.join(DATA_DIR, "files.json")
```

```
        self.quizzes_file = os.path.join(DATA_DIR, "quizzes.json")
```

```
        os.makedirs(DATA_DIR, exist_ok=True)
```

```
        self.load_data()
```

```
    def load_data(self):
```

```
        """o o i . i o o i o (JSON i . o o - i ' )"""
```

```
        if self.use_supabase:
```

```
            return # Supabase o i / i E o i ... o i o o i . i o o %... E o i , i / o f i o i o o o i i i
```

```
        # JSON i . o o - o " o o
```

```
        if os.path.exists(self.folders_file):
```

```
            with open(self.folders_file, 'r', encoding='utf-8') as f:
```

```
                self._folders = json.load(f)
```

```
        else:
```

```
            self._folders = {}
```

```
        if os.path.exists(self.files_file):
```

```
            with open(self.files_file, 'r', encoding='utf-8') as f:
```

```
                self._files = json.load(f)
```

```
        else:
```

```
            self._files = {}
```

```
        if os.path.exists(self.quizzes_file):
```

```
            with open(self.quizzes_file, 'r', encoding='utf-8') as f:
```

```
                self._quizzes = json.load(f)
```

Server/server_data_supabase.py

```
self._quizzes = {}

def save_data(self):
    """
    Save data to JSON file.
    """
    if self.use_supabase:
        return # Supabase

    # JSON file
    with open(self.folders_file, 'w', encoding='utf-8') as f:
        json.dump(self._folders, f, ensure_ascii=False, indent=2)
    with open(self.files_file, 'w', encoding='utf-8') as f:
        json.dump(self._files, f, ensure_ascii=False, indent=2)
    with open(self.quizzes_file, 'w', encoding='utf-8') as f:
        json.dump(self._quizzes, f, ensure_ascii=False, indent=2)

# ==
def get_user_by_email_password(self, email, password):
    """
    Get user by email and password.
    """
    if self.use_supabase:
        try:
            result = self.supabase.table('Folder').select('*').eq('email', email).eq('password', password).execute()
            if result.data and len(result.data) > 0:
                return result.data[0]
            return None
        except Exception as e:
            print(f"Error: {e}")
            return None
    else:
        # JSON file
        for folder_id, folder_data in self._folders.items():
            if folder_data.get('email') == email and folder_data.get('password') == password:
                return folder_data
        return None

def get_user_folders(self, email):
    """
    Get user folders.
    """
    if self.use_supabase:
        try:
            result = self.supabase.table('Folder').select('*').eq('email', email).execute()
            return result.data if result.data else []
        except Exception as e:
            print(f"Error: {e}")
            return []
    else:
        folders = []
        for folder_id, folder_data in self._folders.items():
            if folder_data.get('email') == email:
                folders.append(folder_data)
        return folders

def create_user_with_default_folder(self, username, email, password, default_folder_title=""):
    """
    Create user with default folder.
    """
    if self.use_supabase:
```

Server/server_data_supabase.py

```

# Ɛ, ° ±, í . ° ì ì - (p_folder_id ° NULL)
result = self.supabase.table('Folder').insert({
    'p_folder_id': None,
    'title': default_folder_title,
    'user_name': username,
    'email': email,
    'password': password
}).execute()
if result.data:
    return result.data[0]
return None
except Exception as e:
    print(f"ì <ì 'ì ì ì - ì /°¥ : {e}")
    import traceback
    traceback.print_exc()
    return None
else:
    # JSON í . ° - ° "¤ °
    folder_id = str(len(self._folders) + 1)
    folder_data = {
        'folder_id': folder_id,
        'p_folder_id': None,
        'title': default_folder_title,
        'user_name': username,
        'email': email,
        'password': password,
        'created_at': datetime.now().isoformat()
    }
    self._folders[folder_id] = folder_data
    self.save_data()
    return folder_data

# === Folder Ɛ. ° ¤ ° ' ì ° ===

def get_folder(self, folder_id):
    """í . ° ì ; ° í """
    if self.use_supabase:
        try:
            result = self.supabase.table('Folder').select('*').eq('folder_id', folder_id).execute()
            return result.data[0] if result.data else None
        except Exception as e:
            print(f"í . ° ì ; ° í ì /°¥ : {e}")
            return None
    else:
        return self._folders.get(str(folder_id))

def create_folder(self, p_folder_id, title, user_name, email, password):
    """í . ° ì ì - """
    if self.use_supabase:
        try:
            result = self.supabase.table('Folder').insert({
                'p_folder_id': p_folder_id,
                'title': title,
                'user_name': user_name,

```

Server/server_data_supabase.py

```
        'password': password
    }).execute()
    return result.data[0] if result.data else None
except Exception as e:
    print(f"İşlem başarısız: {e}")
    return None
else:
    folder_id = str(len(self._folders) + 1)
    folder_data = {
        'folder_id': folder_id,
        'p_folder_id': p_folder_id,
        'title': title,
        'user_name': user_name,
        'email': email,
        'password': password,
        'created_at': datetime.now().isoformat()
    }
    self._folders[folder_id] = folder_data
    self.save_data()
    return folder_data

def update_folder(self, folder_id, **kwargs):
    """İşlem: folder_id ile ilgili verileri güncelle"""
    if self.use_supabase:
        try:
            result = self.supabase.table('Folder').update(kwargs).eq('folder_id', folder_id).execute()
            return result.data[0] if result.data else None
        except Exception as e:
            print(f"İşlem başarısız: {e}")
            return None
    else:
        folder_id_str = str(folder_id)
        if folder_id_str in self._folders:
            self._folders[folder_id_str].update(kwargs)
            self.save_data()
            return self._folders[folder_id_str]
        return None

def delete_folder(self, folder_id):
    """İşlem: folder_id ile ilgili verileri sil"""
    if self.use_supabase:
        try:
            result = self.supabase.table('Folder').delete().eq('folder_id', folder_id).execute()
            return True
        except Exception as e:
            print(f"İşlem başarısız: {e}")
            return False
    else:
        folder_id_str = str(folder_id)
        if folder_id_str in self._folders:
            del self._folders[folder_id_str]
            self.save_data()
            return True
        return False
```

Server/server_data_supabase.py

```
# == File ¶ ° ¢ ' ¨ ° ==

def get_file(self, file_id):
    """í ¨ ... ¨ ¨í ""
    if self.use_supabase:
        try:
            result = self.supabase.table('File').select('*').eq('file_id', file_id).execute()
            return result.data[0] if result.data else None
        except Exception as e:
            print(f"í ¨ ... ¨ ¨í ¨ /°¥ : {e}")
            return None
    else:
        return self._files.get(str(file_id))

def get_files_by_folder(self, folder_id=None):
    """í ¨ ... °'° ¨ ¨í°í (° ¢ ¨ ... ¨ ¨ ¨ ' ¨ ¨ í ° ¨ ¨ í , - folder_id° ¨ ¨ í ¨ ¨ ¨ <í >)"
    if self.use_supabase:
        try:
            if folder_id is None:
                # folder_id° ¨ None ¨ .°' . ° ¢ ¨ ¨ í ¨ ... ¨ ¨í
                result = self.supabase.table('File').select('*').order('created_at', desc=True).order_by('created_at', desc=True).limit(10).execute()
            else:
                result = self.supabase.table('File').select('*').eq('folder_id', folder_id).order_by('created_at', desc=True).limit(10).execute()
            return result.data if result.data else []
        except Exception as e:
            print(f"í ¨ ... °'° ¨ ¨ ¨í ¨ ¨í°í ¨ /°¥ : {e}")
            return []
    else:
        files = []
        for file_id, file_data in self._files.items():
            if folder_id is None or file_data.get('folder_id') == folder_id:
                files.append(file_data)
        return files

def get_all_files(self):
    """° ¢ ¨ ¨ í ¨ ... ¨ ¨í°í (° ¢ ¨ ... ¨ ¨ ¨ ' ¨ ¨ í ° ¨ ¨ í , )"
    return self.get_files_by_folder(folder_id=None)

def create_file(self, folder_id=None, title=None, file_path=None, convert_file_path=None, summary_file_path=None):
    """í ¨ ... ¨ ¨ ¨ - (° ¢ ¨ ... ¨ ¨ ¨ ¨ ¨ ' ¨ ¨ í ° ¨ ¨ í , - folder_id° ¨ ¨ í ¨ ¨ ¨ <í >)"
    if self.use_supabase:
        try:
            file_data = {
                'title': title
            }
            if folder_id is not None:
                file_data['folder_id'] = folder_id
            if file_path:
                file_data['file_path'] = file_path
            if convert_file_path:
                file_data['convert_file_path'] = convert_file_path
            if summary_file_path:
                file_data['summary_file_path'] = summary_file_path
```

Server/server_data_supabase.py

```
result = self.supabase.table('File').insert(file_data).execute()
return result.data[0] if result.data else None
except Exception as e:
    print(f"İşlem sırasında hata: {e}")
    import traceback
    traceback.print_exc()
    return None
else:
    file_id = str(len(self._files) + 1)
    file_data = {
        'file_id': file_id,
        'folder_id': folder_id,
        'title': title,
        'file_path': file_path,
        'convert_file_path': convert_file_path,
        'summary_file_path': summary_file_path,
        'created_at': datetime.now().isoformat()
    }
    self._files[file_id] = file_data
    self.save_data()
    return file_data

def update_file(self, file_id, **kwargs):
    """İşlem sırasında hata: {e}"""
    if self.use_supabase:
        try:
            result = self.supabase.table('File').update(kwargs).eq('file_id', file_id).execute()
            return result.data[0] if result.data else None
        except Exception as e:
            print(f"İşlem sırasında hata: {e}")
            return None
    else:
        file_id_str = str(file_id)
        if file_id_str in self._files:
            self._files[file_id_str].update(kwargs)
            self.save_data()
            return self._files[file_id_str]
        return None

def delete_file(self, file_id):
    """İşlem sırasında hata: {e}"""
    if self.use_supabase:
        try:
            result = self.supabase.table('File').delete().eq('file_id', file_id).execute()
            return True
        except Exception as e:
            print(f"İşlem sırasında hata: {e}")
            return False
    else:
        file_id_str = str(file_id)
        if file_id_str in self._files:
            del self._files[file_id_str]
            self.save_data()
            return True
        return False
```

Server/server_data_supabase.py

```
# == Quiz ¶. ° ¢ °' ì ° ==

def get_quiz(self, quiz_id):
    """í .ì f ì i°í """
    if self.use_supabase:
        try:
            result = self.supabase.table('Quiz').select('*').eq('quiz_id', quiz_id).execute()
            return result.data[0] if result.data else None
        except Exception as e:
            print(f"í .ì f ì i°í ì /°¥ : {e}")
            return None
    else:
        return self._quizzes.get(str(quiz_id))

def get_quizzes_by_file(self, file_id):
    """í ì ...ì ì í í .ì f °""°i ì i°í """
    if self.use_supabase:
        try:
            result = self.supabase.table('Quiz').select('*').eq('file_id', file_id).execute()
            return result.data if result.data else []
        except Exception as e:
            print(f"í .ì f °""°i ì i°í ì /°¥ : {e}")
            return []
    else:
        quizzes = []
        for quiz_id, quiz_data in self._quizzes.items():
            if quiz_data.get('file_id') == file_id:
                quizzes.append(quiz_data)
        return quizzes

def create_quiz(self, file_id, question, example=None, correct_answer=None, explanation=None, user_answer=None):
    """í .ì f ì ì -"""
    if self.use_supabase:
        try:
            quiz_data = {
                'file_id': file_id,
                'question': question,
                'correct_is': correct_is
            }
            if example:
                quiz_data['example'] = example
            if correct_answer:
                quiz_data['correct_answer'] = correct_answer
            if explanation:
                quiz_data['explanation'] = explanation
            if user_answer:
                quiz_data['user_answer'] = user_answer

            result = self.supabase.table('Quiz').insert(quiz_data).execute()
            return result.data[0] if result.data else None
        except Exception as e:
            print(f"í .ì f ì ì - ì /°¥ : {e}")
            import traceback
            traceback.print_exc()
```

Server/server_data/supabase.py

```
    else:
        quiz_id = str(len(self._quizzes) + 1)
        quiz_data = {
            'quiz_id': quiz_id,
            'file_id': file_id,
            'question': question,
            'example': example,
            'correct_answer': correct_answer,
            'explanation': explanation,
            'user_answer': user_answer,
            'correct_is': correct_is,
            'created_at': datetime.now().isoformat()
        }
        self._quizzes[quiz_id] = quiz_data
        self.save_data()
        return quiz_data

def update_quiz(self, quiz_id, **kwargs):
    """í .í f ì ° òí .í ,"""
    if self.use_supabase:
        try:
            result = self.supabase.table('Quiz').update(kwargs).eq('quiz_id', quiz_id).execute()
            return result.data[0] if result.data else None
        except Exception as e:
            print(f"í .í f ì ° òí .í , ì /°¥ : {e}")
            return None
    else:
        quiz_id_str = str(quiz_id)
        if quiz_id_str in self._quizzes:
            self._quizzes[quiz_id_str].update(kwargs)
            self.save_data()
            return self._quizzes[quiz_id_str]
        return None

def delete_quiz(self, quiz_id):
    """í .í f ì >ì """
    if self.use_supabase:
        try:
            result = self.supabase.table('Quiz').delete().eq('quiz_id', quiz_id).execute()
            return True
        except Exception as e:
            print(f"í .í f ì >ì ì /°¥ : {e}")
            return False
    else:
        quiz_id_str = str(quiz_id)
        if quiz_id_str in self._quizzes:
            del self._quizzes[quiz_id_str]
            self.save_data()
            return True
        return False

# == í ,í ì -ì ì í °' ì ° (E,°ì ì . ì% ° ì ì í ,í ) ==

def get_user(self, email):
```



```

Server/serven_data/supabase.py Folder í ì .°, ì ì ì†« °† ì$, í .° °° í )"""
    folders = self.get_user_folders(email)
    return folders[0] if folders else None

def create_user(self, username, email, password):
    """ì <ì 'ì ì ì - (í ,í ì - - £,°°†, í .° ì í » ì ì -)"""
    return self.create_user_with_default_folder(username, email, password)

def get_folders(self, user_email):
    """ì <ì 'ì ì ì í .° °°'°i ì i°í (í ,í ì -)"""
    folders = self.get_user_folders(user_email)
    return [f['title'] for f in folders] if folders else []

def create_folder(self, user_email, folder_name):
    """í .° ì ì - (í ,í ì - - £,°ìj. ì <ì 'ì ì ì °†. í ì )"""
    # £,°ìj. ì <ì 'ì ì ì í .° °¥... ì°¼ ì user_name£†... password £° ì ,ì /£,°
    user_folders = self.get_user_folders(user_email)
    if not user_folders:
        return False

    first_folder = user_folders[0]
    return self.create_folder(
        p_folder_id=None,
        title=folder_name,
        user_name=first_folder.get('user_name', ''),
        email=user_email,
        password=first_folder.get('password', '')
    ) is not None

def update_folder_name(self, user_email, old_name, new_name):
    """í .° ì .°f °† £†% (í ,í ì -)"""
    folders = self.get_user_folders(user_email)
    for folder in folders:
        if folder.get('title') == old_name:
            return self.update_folder(folder['folder_id'], title=new_name) is not None
    return False

def delete_folder_by_name(self, user_email, folder_name):
    """í .° ì >ì (í ,í ì -)"""
    folders = self.get_user_folders(user_email)
    for folder in folders:
        if folder.get('title') == folder_name:
            return self.delete_folder(folder['folder_id'])
    return False

```